# 关于Detours/Minhook挂钩引擎的UnHook

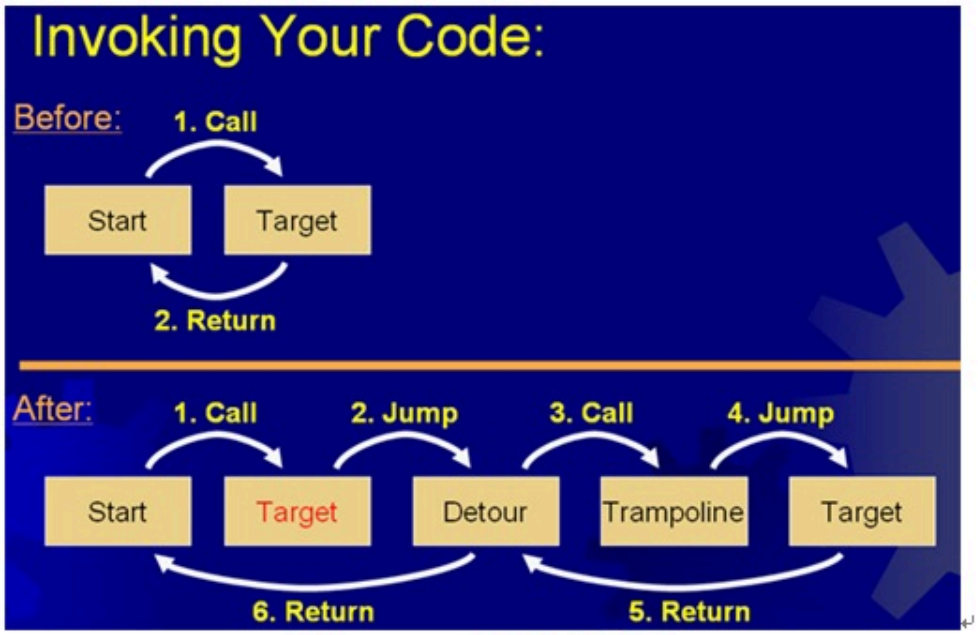
转载 liqiang981 于 2016-07-13 15:01:00 发布 阅读量1.1k 收藏 2 点赞数

**关于Detours/Minhook挂钩引擎的UnHook**

文章本身可能用处不大，很少有人会用到所说的这些，权当积累。

一直在使用Detours 和 MinHook 两个Hook引擎进行一些系统API的挂钩，实现特殊的功能。就如标题所用词，UnHook，很少将一个已经Hook的函数再次进行UnHook，所以一直没有注意到问题所在。同事的提醒，让自己有了一点兴趣看一下这个UnHook 到底有什么问题呢?

首先简单说一下Detours / MinHook的基本原理，就是在要挂钩的函数地址处，置一个跳转指令，直接跳转到Detours函数上。然后再由Detours函数通过Trampoline函数（保存的是被Hook函数地址处被置换为JMP指令的原始指令加上一个JMP指令（用于跳转到原始函数））调用原始函数。在依次返回。这样就实现了Detours函数的调用。可以在Detours函数中实现一些特殊的逻辑，满足特殊需求。下面的图是从其他地方借鉴过来的，可以参考一下，非常清晰地说明了整个过程。




（图 2: Detour函数的调用过程）

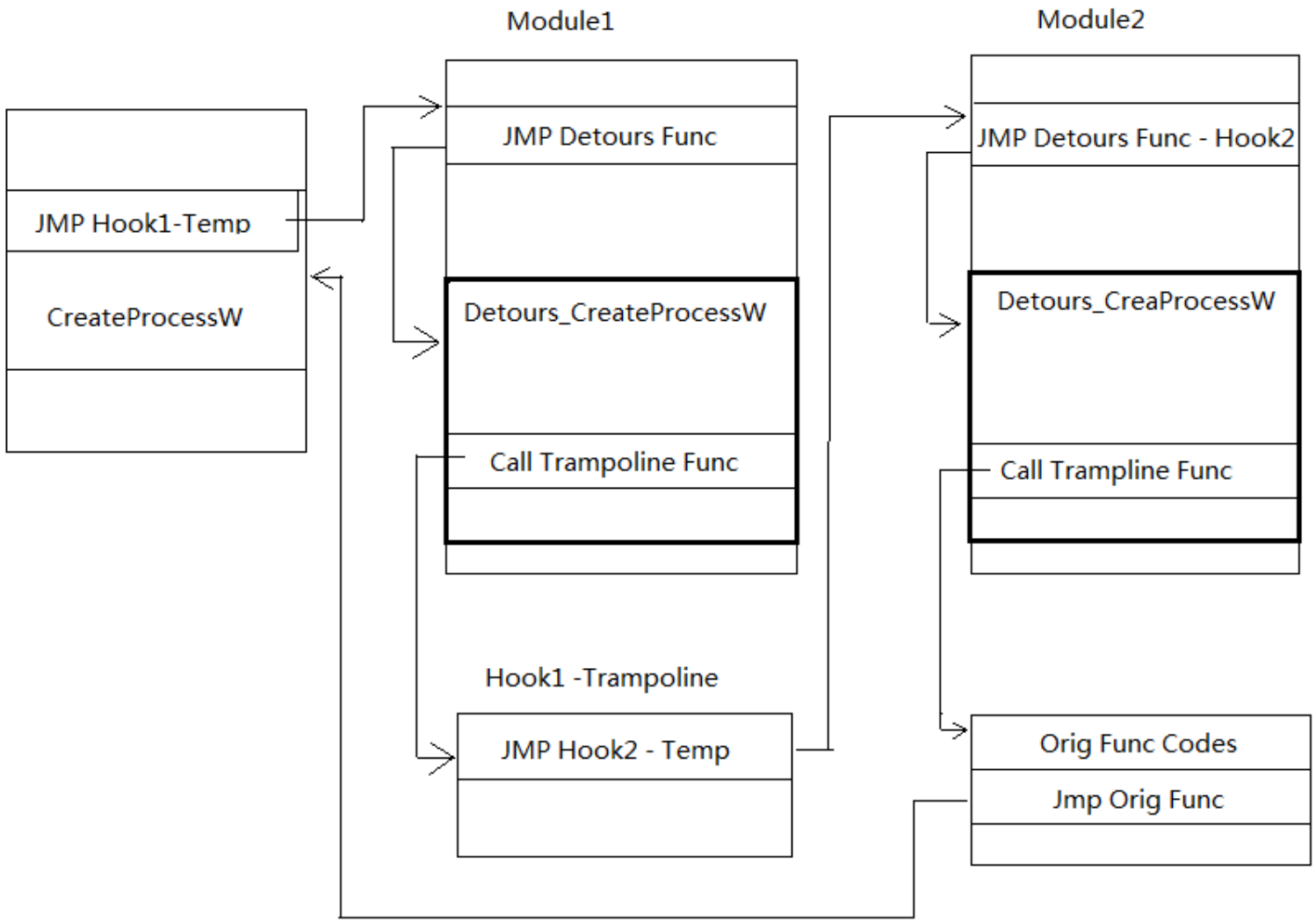
那UnHook是什么样一个过程呢？根据上图就很简单了，其实就是将复制到Trampoline中的原函数指令，重新复制回Target函数中。也即覆盖掉Target函数被Hook之后目标地址上的那个JMP 指令。

当在一个进程中，只有一个模块在一个目标函数上（比如CreateProcessW）挂钩子时，对这个钩子UnHook很正常。UnHook后，目标函数（Target）恢复正常，变为Kernel32.dll 模块被加载时的样子。

但是！（人生中最怕的就是“但是”，不是么？总有意想不到的东西存在。）在这个纷繁复杂的世界中，“撞车”的情况再正常不过了，微信，陌陌，手机QQ。。。，全都撞在社交这个车上了。一个进程中，你有模块在挂CreateProcessW的钩子，别人就不可以有模块挂吗？除了安全软件霸道地将人拒之门外之外，很多进程大家还都是公平的，你挂，我也挂。你摘，我也摘。要把这些全都说清楚，得写一天的文章，也不一定说完。类似Detours库的开源库就有一堆，还有自己实现的。Detours修改了五个字节，有的修改八个字节，各有各理，不去评判孰是孰非。

文章只说用Detours库Hook多次，或者用MinHook挂多次钩子的情况。Detours是微软，不能再官方了，稳定性没得说（正规军也能碰上打游击的，碰上了打游击的就没什么稳定性可言了）。MinHook是开源的，用的HDE反汇编引擎分析指令，基本原理看过，没仔细抠细节，基本确定稳定，且细节处理也到位。

多次Hook，同一个模块中不可能了，拿MinHook来说，它的全局变量中保存了已经挂了的钩子的信息，对同一个地址只能Hook一次，但是同一个进程多个模块就不同了。多次Hook的时候，自然就会出现Hook链了（这就是Hook引擎的好处）。Hook链中依次跳到各个模块对目标函数Hook的Detours函数中，执行逻辑。与单个Hook时类似，依据不同的模块Hook的先后顺序，可以形成如下图所示链。




很明显，模块2是先挂的钩子，它将CreateProcessW的其实5字节的指令复制到了它的Trampoline函数中，并且添加了一个JMP 到CreateProcessW+5Byte的地址，以继续执行CreateProcessW之后的指令。Module1再次对CreateProcessW挂钩子时，它将Module2写的JMP到 Module2地址的指令复制到了Module1的Trampline 函数中，这样就行了，一个链，所以对同一个函数地址，多次Hook时就形成了Hook链。

那么文章要说的UnHook问题也就很明显了，分为很多种情况。暂且不说具体应用场景是否存在，仅仅做分析而已。这里以图示的两个Hook为例，简单分析一下：

1. 如果按照Hook的反顺序，也即先Module1 UnHook钩子，再Module2 UnHook钩子，根据单个钩子的那个UnHook说明，Module1将JMP Hook2-Temp的指令恢复到了CreatProcessW挂钩指令上，那么这时CreateProcessW处就变成了上面所说的单Hook的样子，无论此时调用CreateProcessW，还是继续Module2的UnHook都不会有问题。

2. 如果不按照Hook的反顺序，也即先让Module2 UnHook，那么CreateProcessW被Hook的地方指令被恢复为了原始的情况（Kernel32.dll初始映射的情况）。这时调用CreateProcessW不会出现问题（相当于没有被Hook嘛）。也就是说虽然Module1没有UnHook，但是它的钩子被覆盖掉了，也不会调用到Module1的Detours函数了。

如果此时，Module1 继续执行UnHook，那么很明显 JMP Hook2-Temp 指令被覆盖会了CreateProcessW Hook的五个字节指令上了，而很明显它要跳转到Module2 的Detours函数。在Detours和MinHook中，Trampoline函数所占用的内存空间是被通过堆栈分配的，它随时都有可能释放掉，一旦释放掉，将跳转到无效的内存空间，后果不堪设想（大不了就是崩溃呗）。

再一点，之前已经将Module2 的钩子Unhook掉了，被Module1重写JMP到Module2，显然程序逻辑上也是有问题。

这里分析了最简单的两个Hook时，当有更多的模块，对同一个地址挂接钩子，原理类似，情况更为复杂些。但是，这里想要说的是，无论是否更为复杂，任意地UnHook已经Hook的地址（使用Detours/MinHook的UnHook函数）都是有发生问题的风险的。

根据处理计算机问题的 鸵鸟原理 ，那就不要UnHook，直到进程结束。（当然有特殊要求的必须UnHook，那就另说了，单独处理）

示例源码：http://download.csdn.net/detail/xiao_0429/9119493